

Lab 4 — Vibe Coding Chat History

Software Engineering — Coin Collector Project

06_coin_collector.zip

File

I am doing Lab 4: Vibe Coding for my Software Engineering course.

This is the Coin Collector project assigned to me. First, inspect the complete project and understand how the existing code works.

TASK 1 — FIX THE COIN COLLECTION BUG

The README says there is a deliberate bug: the coin collection check runs every frame, so if the player continues overlapping a coin, the same coin can increase the score repeatedly. The coin is also not properly removed/marked as collected.

Fix ONLY this bug.

Requirements:

1. Each coin must be collected exactly once.
 2. The score must increase only once when a coin is collected.
 3. The collected coin must be removed or marked as collected so it cannot be collected again.
 4. Preserve the existing player movement and game behavior.
 5. Do NOT implement the other README features yet:
 - multiple coin types
 - obstacles
 - 30-second timer/game-over/restart
 6. Inspect the relevant files before modifying anything.
 7. Make the minimum necessary code changes.
 8. After making the changes, explain:
 - which files you changed
 - what was wrong
 - how your fix prevents repeated collection
- ...

Worked for 34s

Task 1 is fixed ■

I inspected the complete project and made only the necessary change.

Changed file

coin-collector/game/game_engine.py

What was wrong

Every frame, update() did:

```
collected = check_collection(self.player, self.coins)
```

```
for coin in collected:  
    self.score += coin.value
```

The collision was detected correctly, but the coin stayed inside self.coins.

So if the player stood on the same coin for 10 frames:

1 coin × 10 frames = 10 points ■

Fix

After awarding the points, collected coins are removed:

```
def update(self):  
    collected = check_collection(self.player, self.coins)  
    for coin in collected:  
        self.score += coin.value  
    self.coins = [coin for coin in self.coins if coin not in collected]
```

Now:

Player touches coin.
Coin is detected.
Its value is added once.
Coin is removed from self.coins.
On subsequent frames, that coin no longer exists, so it cannot award points again. ■
This preserves the existing movement, rendering, collision logic, and game behavior. I did not implement multiple coin types, obstacles, timer, game-over, or restart.

Fixed project
Download the fixed Coin Collector project

I also verified that all Python files compile successfully. The only limitation is that Pygame isn't installed in my execution environment, so I couldn't launch the graphical game itself here.

TASK 2 – IMPLEMENT MULTIPLE COIN TYPES

Now implement ONLY the multiple coin types feature from the README.

Requirements:

1. Add at least three coin types:
 - Bronze coin = 1 point
 - Silver coin = 3 points
 - Gold coin = 5 points
2. Each coin type must be visually distinct.
3. The correct value must be awarded when each type is collected.
4. Keep the Task 1 fix: each coin must still be collectible exactly once.
5. Keep the existing player movement and game behavior.
6. Do NOT implement obstacles.
7. Do NOT implement the 30-second timer, game-over, or restart feature.
8. Make the minimum necessary changes.
9. Inspect the existing code and modify the appropriate files rather than rewriting the project.
10. After making the changes, explain which files you changed and how the three coin types work.

...

Worked for 37s

Task 2 – Multiple Coin Types ■

Implemented only the multiple coin types feature on top of the Task 1 version.

File changed
game/game_engine.py

No changes were needed to coin.py, collection.py, renderer.py, player movement, or main.py because the existing Coin class already supported both value and color.

How the three types work
I added:

Coin Points Color

- Bronze 1 (176, 112, 64)
- Silver 3 (192, 192, 192)
- Gold 5 (230, 190, 60)

When each coin is created, the game randomly selects one of these types:

```
_, value, color = random.choice(COIN_TYPES)
return Coin(x=x, y=y, radius=12, value=value, color=color)
```

The existing scoring code already uses:

```
self.score += coin.value
```

so the correct score is automatically awarded:

Bronze → +1
Silver → +3
Gold → +5

Task 1 fix is preserved

The collection logic still removes collected coins:

```
self.coins = [coin for coin in self.coins if coin not in collected]
```

Therefore, even if the player remains standing on a coin, it can only contribute its value once.

Not implemented
I did not add:

- Obstacles
- 30-second timer
- Game-over
- Restart
- Any changes to player movement

I also ran a Python syntax/static verification successfully.

Download Task 2

Download the completed Coin Collector – Task 2

TASK 3 – IMPLEMENT OBSTACLES

Now implement ONLY the obstacles feature from the README.

Requirements:

1. Add obstacles inside the play area.
2. Obstacles must remain within the game boundaries.
3. When the player touches an obstacle, apply a collision consequence such as losing one life.
4. A single touch should cause only one life loss, not repeatedly decrease lives every frame while the player remains overlapping the obstacle.
5. Add enough obstacles to make the game feature clearly visible and testable.
6. Keep the Task 1 fix: each coin is collected exactly once.
7. Keep Task 2: bronze, silver, and gold coins with values 1, 3, and 5.
8. Preserve existing player movement and game behavior.
9. Do NOT implement the 30-second timer, game-over, or restart feature yet.
10. Make the minimum necessary changes.
11. After implementing, explain which files were changed and how obstacle collision and the single-touch life penalty work.

...

Worked for 44s

Task 3 – Obstacles implemented ■

I built Task 3 on top of the completed Task 2 project and only added the obstacle/lives functionality.

Files changed

game/obstacle.py – new file

Defines a simple rectangular Obstacle.
Stores its rectangle and drawing color.
game/game_engine.py

Creates 4 obstacles inside the play area.
Adds 3 starting lives.
Checks player-obstacle collisions.
Applies the one-touch life penalty.
Keeps the Task 1 coin-removal fix.
Keeps the Task 2 bronze/silver/gold coins.
game/renderer.py

Draws the obstacles as visible red rectangles.
Displays the current number of lives.
How obstacle collision works
There are four obstacles positioned at:

(100, 100) → 150 × 25
(450, 100) → 150 × 25
(100, 350) → 150 × 25
(450, 350) → 150 × 25
All four are completely inside the 700 × 500 game area.

The player starts with:

```
self.lives = 3
```

When the player touches an obstacle:

```
if touching_obstacle and not self._obstacle_collision_active:
    self.lives -= 1
    self._obstacle_collision_active = True
```

The important part is `_obstacle_collision_active`.

Why lives don't decrease every frame
If the player stays inside an obstacle for 30 frames:

1 continuous collision → only 1 life lost ■

The collision becomes inactive only after the player leaves:

```
elif not touching_obstacle:  
self._obstacle_collision_active = False
```

So:

Touch → -1 life → stay overlapping → nothing → leave → touch again → -1 life

This satisfies the single-touch requirement.

Previous tasks remain intact

■ Bronze = 1 point

■ Silver = 3 points

■ Gold = 5 points

Coins are removed after collection → no repeated scoring

Existing arrow-key movement is unchanged

■ No timer

■ No game-over

■ No restart

■ No obstacle movement/complex mechanics

I also verified the Python files compile and tested the obstacle boundary and one-touch collision logic.

Download Task 3

Download the completed Coin Collector – Task 3

TASK 4 – IMPLEMENT 30-SECOND TIMER, GAME OVER AND RESTART

Now implement ONLY Task 4 from the README.

Requirements:

1. Add a 30-second countdown timer.
2. Display the remaining time clearly during gameplay.
3. When the timer reaches zero, stop the round.
4. If the player's lives reach zero, stop the round.
5. When the round ends, display the final score clearly.
6. Provide a restart mechanism.
7. Restart must reset:
 - score
 - lives
 - timer to 30 seconds
 - coins
 - obstacles/game state as appropriate
8. Preserve all previous features:
 - Task 1: each coin is collected exactly once
 - Task 2: bronze = 1, silver = 3, gold = 5
 - Task 3: obstacles cause one life loss per touch
9. Do not remove or break existing player movement.
10. Make the minimum necessary changes.
11. Do not rewrite unrelated parts of the project.
12. After implementing, explain which files were changed and how the timer, game-over, and restart work.

...

Worked for 44s

Task 4 – Timer, Game Over & Restart ■

Implemented only Task 4 on top of your completed Task 3 project.

Files changed

1. game/game_engine.py

Added the 30-second timer.

Added round/game-over state.

Stops movement and game updates when the round ends.

Detects both:

timer reaching 0

lives reaching 0

Displays the final score.

Added restart/reset logic.

Preserved all previous coin and obstacle behavior.

2. main.py

Updated the controls comment to document the R restart key.

No unrelated files were rewritten.

1. 30-second countdown
The game starts with:

```
ROUND_DURATION = 30
```

The engine tracks elapsed time and calculates:

```
self.remaining_time = max(0, ROUND_DURATION - elapsed)
```

The HUD displays:

```
Score: 0
Lives: 3
Time: 29
and counts down during gameplay.
```

2. Timer reaches zero
When the timer reaches zero:

```
if self.remaining_time <= 0:
    self.game_over = True
    return
```

The round stops, so the player can no longer move or collect coins.

3. Lives reach zero
The existing Task 3 obstacle collision remains intact.

When a valid obstacle touch occurs:

```
self.lives -= 1
```

The existing collision latch still ensures that continuously touching the same obstacle costs only one life.

When lives reach zero:

```
if self.lives <= 0:
    self.lives = 0
    self.game_over = True
```

The round then stops.

4. Game-over screen
When the round ends, the game displays:

```
GAME OVER - Final Score: 15 - Press R to Restart
So the final score is clearly visible.
```

5. Restart
Press R after game over.

The restart calls:

```
self._reset_round()
```

which resets:

```
Score → 0
Lives → 3
Timer → 30 seconds
Player → starting position
Coins → new set of coins
Obstacles → reset
Collision state → reset
Game-over state → active
So you get a completely fresh round.
```

Previous tasks preserved

■ Task 1: Coins can only be collected once.

■ Task 2:

Bronze → 1 point
Silver → 3 points
Gold → 5 points
■ Task 3:

Four visible obstacles
Obstacles stay inside the play area
Collision costs one life
Continuous overlap does not repeatedly remove lives
■ Player movement preserved.

■ No unrelated features added.

Download
Download the completed Task 4 Coin Collector project

The Python files also passed compilation verification.